

Prueba técnica — Ingeniería de Datos

Gracias por tomarte el tiempo de resolver esto. Armamos la prueba con problemas parecidos a los que aparecen en el día a día del equipo: datos que llegan desde Mongo, un bug que anda duplicando filas en producción, y una carga incremental que no debería romperse si llega un dato viejo por error.

Cuando termines, mándanos tu código (repo, notebook exportado, o el .py directo) junto con las respuestas escritas de los ejercicios 2 y 3.

Un par de cosas antes de empezar

- Trabajamos con PySpark. Los imports que probablemente necesites: `pyspark.sql.functions`, `pyspark.sql.types`, `Window`, `DeltaTable`.
- Usamos sufijos según el tipo de dato en los nombres de columna: `_id`, `_cd`, `_txt`, `_val`, `_dec`, `_flag`, `_arr`, `_dict`, `_ts`, `_dt`, `_int`.
- Comenta tus supuestos directo en el código — no hace falta un documento aparte explicando cada decisión.
- No necesitas correrlo en un cluster real ni que sea perfecto; nos importa la lógica y cómo manejas los casos raros.
- Como en el resto del código del equipo: comentarios en español.

Ejercicio 1 — De Bronze a Silver

Este es un extracto de lo que llega desde MongoDB a la capa Bronze: la colección customers, con sus arrays anidados, sus nulls sueltos y un par de inconsistencias típicas de un sistema de origen que no valida mucho antes de guardar.

```
[
  {
    "_id": "60c1f2a1e4b0a1a2b3c4d5e6",
    "country": "GT",
    "full_name": "María López",
    "contact": {
      "phones": ["+50255512345", "+50255598765"],
      "email": "maria.lopez@example.com"
    },
    "created_at": "2024-03-11T14:22:00Z",
    "is_active": "true",
    "purchases": [
      {"order_id": "ORD-001", "amount": 129.5, "currency": "GTQ"},
      {"order_id": "ORD-002", "amount": 45.0, "currency": "GTQ"}
    ]
  },
  {
    "_id": "60c1f2a1e4b0a1a2b3c4d5e7",
    "country": "SV",
    "full_name": "Carlos Reyes",
    "contact": { "phones": [], "email": null },
    "created_at": null,
    "is_active": 1,
    "purchases": []
  },
  {
    "_id": "60c1f2a1e4b0a1a2b3c4d5e8",
    "country": "ec",
    "full_name": "Ana Torres",
    "contact": {
      "phones": ["+593987654321"],
      "email": "ana.torres@example.com"
    },
    "created_at": "2024-03-12 09:15:00",
    "is_active": "false",
    "purchases": [
      {"order_id": "ORD-003", "amount": 88.25, "currency": "USD"}
    ]
  },
  {
    "_id": "60c1f2a1e4b0a1a2b3c4d5e9",
    "country": "PE",
    "full_name": null,
    "contact": {
      "phones": ["+51987654321"],
      "email": "sin_nombre@example.com"
    },
    "created_at": "2024-03-13T08:00:00Z",
    "is_active": "true",
    "purchases": [
      {"order_id": "ORD-004", "amount": 200.0, "currency": "PEN"},
      {"order_id": "ORD-005", "amount": null, "currency": "PEN"}
    ]
  }
]
```

Con esto, arma la transformación a Silver. Concretamente:

1. Carga los datos (puedes leerlos como JSON directamente, o construir el DataFrame a mano, como prefieras).
2. Pasa a esquema Silver con la convención de sufijos: customer_id, pais_cd (ojo con el "ec" en minúscula), nombre_txt, email_txt, telefonos_arr, creado_ts, activo_flag como booleano real.
3. Saca las compras (purchases) a su propia tabla, customer_purchases_silver, sin perder la referencia a customer_id y pais_cd.
4. Decide qué hacer con cada null que encuentres (full_name, created_at, email, un amount nulo dentro de una compra, un array de phones vacío) y déjalo explícito en el código
5. Particiona el resultado por país y fecha.

Ejercicio 2 — Hay un bug en producción

Un compañero escribió esta función para el merge incremental de orders_silver. Funciona la mayoría del tiempo, pero de vez en cuando aparecen duplicados en algunos países y el equipo de infraestructura se ha quejado de que el job tarda cada vez más.

```
import pyspark.sql.functions as F
from delta.tables import DeltaTable

def merge_orders(spark, new_batch_df, target_table_path):
    """
    Hace merge incremental de nuevas órdenes a la tabla Silver.
    new_batch_df: order_id_txt, customer_id, pais_cd, monto_dec, updated_ts
    """
    target = DeltaTable.forPath(spark, target_table_path)

    # Enriquemos con datos del cliente
    customers_df = spark.table("customers_silver")
    enriched_df = new_batch_df.join(
        customers_df,
        new_batch_df.customer_id == customers_df.customer_id,
        "left"
    )

    # Bug intencional #1
    merge_keys = "order_id_txt"

    (target.alias("t")
     .merge(
         enriched_df.alias("s"),
         f"t.{merge_keys} = s.{merge_keys}"
     )
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute()
    )

    # Bug intencional #2 (particionamiento)
    enriched_df.write.format("delta") \
        .mode("append") \
        .partitionBy("updated_dt") \
        .save(target_table_path + "_backup")

    return enriched_df
```

Dos datos que te van a servir: `customers_silver` todavía tiene clientes duplicados por `customer_id` en algunos países (viene de un histórico que no se ha limpiado), y el mismo `order_id_txt` se repite entre países distintos porque cada uno viene de un sistema de origen diferente que no coordina sus IDs con los demás.

Antes de tocar el código, contesta por escrito:

6. ¿Qué pasa exactamente cuando dos países generan el mismo `order_id_txt` con las `merge_keys` actuales?
7. ¿Por qué el join contra `customers_silver` puede estar multiplicando filas, y en qué momento lo arreglarías?
8. ¿Qué problema concreto trae particionar solo por `updated_dt` cuando el equipo opera en 9 países?
9. ¿cuándo SÍ sería seguro usar `whenMatchedUpdateAll()` tal cual, sin ninguna condición extra?

Con eso claro, reescribe la función con los tres problemas corregidos.

Ejercicio 3 — Carga incremental que no se rompa con datos viejos

Necesitamos cargar `inventory_silver` de forma incremental usando `updated_ts` como watermark.

Tabla histórica (`inventory_silver` actual):

producto_id	pais_cd	stock_int	updated_ts
P-100	GT	45	2024-05-01 08:00:00
P-101	GT	12	2024-05-01 08:00:00
P-100	HN	20	2024-05-01 08:00:00
P-102	AR	8	2024-04-30 17:30:00

Batch que llega hoy (`inventory_bronze_batch`):

producto_id	pais_cd	stock_int	updated_ts
P-100	GT	40	2024-05-02 09:10:00
P-103	GT	5	2024-05-02 09:12:00
P-100	HN	20	2024-04-29 10:00:00
P-102	AR	3	2024-05-02 10:00:00

Fíjate bien en la fila de P-100/HN en el batch — no es un error de dedo, es intencional.

10. Define la configuración (`merge_keys`, columna de watermark) en un diccionario Python.
11. Escribe `incremental_load(spark, batch_df, target_table_path, conf)`: que revise si la tabla ya existe (si no, la crea), que respete el watermark para no pisar datos con algo más viejo, que inserte lo nuevo, y actualice solo cuando corresponde.
12. Dinos cómo queda `inventory_silver` después de correr tu función contra estos datos — las 4 filas originales más la nueva.

Eso es todo. Si algo no queda claro, mejor pregunta antes de asumir — también es parte de cómo trabajamos.